UI-UG 2: Advancing Mobile UI Understanding and Generation with Fine-Grained Supervision and Reinforcement Learning

Hao Yang **Shuoyang Liu** **Xiaoyu Lin**
Zhou Fang **Maji Huang** **Zhaosong Huang**
Ruichao Mao **Teng Guo** **Hai Rao**

Ant Group

Abstract

Mobile UI understanding and generation require both fine-grained component perception and reliable structured synthesis. Inspired by the unified direction introduced by UI-UG [46], we present **UI-UG 2**, a compact 4B-parameter multimodal large language model (MLLM) that advances mobile **UI U**nderstanding and **G**eneration with broader task coverage, a front-end-aligned taxonomy, and online render-aware reinforcement learning. UI-UG 2 constructs a multi-source understanding corpus from RICO, VINS, Android in the Wild, and Android Control, annotated with an 80-category two-level UI taxonomy aligned with front-end component semantics, and pairs it with over 5,000 requirement–screenshot–JSON tuples for progressive structured UI generation through a domain-specific language (DSL). For understanding, UI-UG 2 introduces decomposed rewards for category prediction, OCR fidelity, color recognition, dense grounding, and GUI grounding. Unlike the offline Direct Preference Optimization (DPO)-style generation optimization in UI-UG, UI-UG 2 optimizes generation through end-to-end online Group Relative Policy Optimization (GRPO): sampled DSL outputs are rendered with a Playwright-based server-side pipeline and evaluated using LLM-as-a-Judge and contrastive preference ranking. Experiments show that UI-UG 2 achieves the strongest referring and dense-grounding results among the evaluated models on our 80-category benchmark, and achieves the best composite generation score among compared models at a compact scale (4B). More importantly, we observe a consistent understanding-to-generation transfer: adding fine-grained UI perception supervision improves generation quality under a fixed training budget, suggesting that component-level semantics provide a shared representational foundation for both tasks.

1 Introduction

Mobile user interfaces are the primary bridge between users and digital services. As applications grow in complexity, the ability to accurately *understand* and *generate* UIs underpins a broad spectrum of downstream applications, from screen parsing and GUI agent interaction to personalized interface synthesis and LLM-driven front-end development. These two capabilities, however, have largely evolved along separate tracks, with separate models, separate data pipelines, and separate evaluation protocols.

General-purpose MLLMs such as GPT [32], Claude [2], Gemini [13], and Qwen-VL [3] can nominally perform both tasks, yet their accuracy remains insufficient for precision-critical mobile scenar-

Preprint.

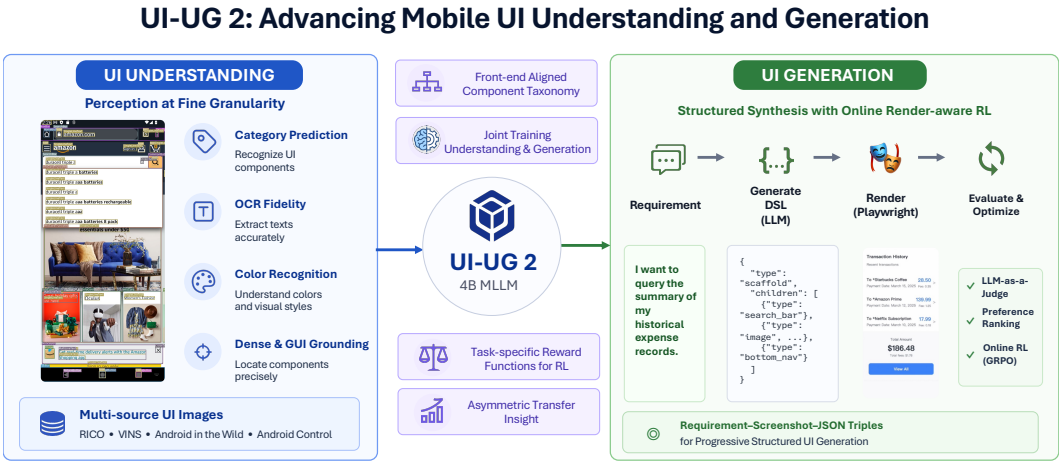


Figure 1: Overview of UI-UG 2. UI-UG 2 unifies fine-grained mobile UI understanding and structured UI generation with a front-end-aligned taxonomy, joint training, task-specific rewards, and online render-aware optimization.

ios. Mobile screens are fundamentally different from natural images: they are dense compositions of small, semantically loaded components—buttons, icons, text fields, navigation bars—whose interaction semantics require fine-grained categorical reasoning. Distinguishing a *CloseButton* from a *ReturnButton*, or a *SearchBar* from a generic *TextInput*, is essential for reliable automation and high-quality generation, but lies beyond the granularity that general-purpose models reliably achieve. Specialized approaches address one side of the problem—Ferret-UI [48] and Ferret-UI2 [23] for grounding and referring, DCGen [41] and DeclarUI [52] for structured generation—while UI-UG [46] took an initial step toward a unified model. However, important limitations remain: its labels were not explicitly organized around front-end component semantics, its understanding coverage focused mainly on referring and dense grounding, its generation optimization relied on offline Direct Preference Optimization (DPO) over a fixed preference dataset, and its evidence for understanding-to-generation transfer remained preliminary.

We present **UI-UG 2**, a compact 4B MLLM for mobile UI Understanding and Generation, with an overview illustrated in Figure 1. UI-UG 2 keeps the central goal of unified UI understanding and generation while strengthening the data, training, and analysis pipeline. On the data side, we construct a large-scale understanding benchmark from four public sources (RICO, VINS, Android in the Wild, and Android Control), covering over 30,000 screenshots with 80 fine-grained element categories organized in a two-level hierarchy aligned to front-end conventions, enriched with element-level, action-level, OCR, and color annotations. We pair this with a curated generation corpus of over 5,000 requirement–screenshot–JSON triples whose compact domain-specific language (DSL) supports cross-platform rendering and progressive streaming.

On the optimization side, we adopt a two-stage pipeline of supervised fine-tuning followed by task-specific Group Relative Policy Optimization (GRPO). For understanding, we design decomposed rewards targeting format compliance, category prediction, OCR fidelity, color recognition, dense grounding, and GUI grounding. For generation, UI-UG 2 performs online render-aware RL: each sampled DSL is rendered by a Playwright-based server-side pipeline, then evaluated through LLM-as-a-Judge and Contrastive Preference Ranking (CPR). This closes the loop between structured generation, actual rendering, and reward feedback, making optimization better aligned with deployment-time UI quality.

Experiments demonstrate that UI-UG 2 achieves the strongest referring and dense-grounding results among the compared models on our benchmark, strong GUI grounding on ScreenSpot mobile benchmarks, and the best overall generation quality among compared models at a compact 4B scale. Through data-mixing ablations, reward transfer analysis, and attention probing, we further observe an asymmetric synergy: *understanding supervision improves generation quality at an appropriate data-mixing ratio*. This finding—that component-level perceptual representations form a

transferable foundation for generative tasks—offers a concrete design principle for future mobile UI systems.

Our contributions are:

- **UI-UG 2.** We propose UI-UG 2, a compact unified MLLM that jointly supports mobile UI understanding and generation, advancing the UI-UG direction [46] with broader task coverage, stronger efficiency, and more rigorous optimization. UI-UG 2 achieves the best results among the compared models on our newly constructed referring and dense-grounding benchmark, and obtains the best composite generation score among all compared baselines despite using only a compact 4B backbone.
- **Front-end-aligned data and tasks.** We contribute an 80-category hierarchical UI understanding dataset with taxonomy aligned to front-end conventions, supporting referring, dense grounding, and GUI grounding, together with a high-quality UI generation corpus based on a compact, renderable JSON DSL.
- **Task-specific online GRPO.** We introduce decomposed reward functions for category, OCR, color, dense grounding, GUI grounding, format validity, renderability, LLM-judged UI quality, and CPR-based preference, enabling online render-aware optimization for generation.
- **Understanding-to-generation synergy.** We provide systematic evidence that fine-grained UI perception consistently benefits generation quality, supported by data-mixing ablations, reward transfer analysis, and attention probing.

2 Related Work

UI Understanding. UI understanding is central to screen parsing, interface analysis and intelligent automation. Early advances were enabled by datasets such as VINS [5], ERICA [10], WebSRC [7], and RICO [11]. Methodologically, the field has evolved from relying on structured signals such as Document Object Models (DOMs) or view hierarchies [6] to parsing interfaces directly from pixels [43, 20, 27, 28]. More recently, MLLM-based models such as Ferret-UI [48] and Ferret-UI2 [23] have unified multiple UI understanding tasks, while agent-oriented grounding models such as SeeClick [8], ShowUI [24], UI-TARS [33], GUI-Owl [47] and UI-Venus [14] have improved fine-grained localization on benchmarks such as ScreenSpot Series [8, 21]. Compared with prior work, we introduce a richer front-end-aligned taxonomy and support a more comprehensive set of fine-grained UI understanding tasks.

UI Generation. Automated UI generation aims to translate visual designs or user requirements into executable interface code. Early approaches such as pix2code [4], Sketch2Code [18], and REMAUI [31] focused on image-to-code prediction or explicit component reconstruction. More recent MLLM-based methods, including DCGen [41] and DeclarUI [52], improve structured UI generation through stronger multimodal reasoning. Benchmarks such as Web2Code [50] and LiveCodeBench [17] have also helped standardize evaluation. In contrast, we study MLLM-based UI generation from both user requirements and reference images, with additional support for interactive prototyping.

Unified UI Understanding and Generation. UI-UG [46] first explored jointly training mobile UI understanding and structured UI generation in a single MLLM, using a renderable DSL and task-aware post-training. This line of work suggests that UI perception and UI synthesis can share a common representation, but several questions remain open, including how to define front-end-aligned supervision, how to optimize generation under rendered visual feedback, and how to analyze transfer between understanding and generation more systematically.

Reinforcement Learning and Multi-task MLLMs. Post-training of MLLMs has evolved from RLHF [9] to preference optimization such as DPO [34] and efficient policy optimization such as GRPO [15], with recent multimodal extensions including Visual-RFT [26], R1-VL [51], and VLM-R1 [37]. We apply GRPO to UI understanding and generation with task-specific rewards for structured perception and renderable synthesis. Related multi-task studies show that heterogeneous objectives can cause imbalance and asymmetric transfer [25, 39, 35], motivating our sequential optimization and transfer analysis.

3 Data and Structured Representation

3.1 Understanding Corpus and Annotation

UI-UG 2 uses structured supervision for both UI understanding and generation. For understanding, we curate over **30,000** deduplicated mobile screenshots from RICO [11], VINS [5], Android in the Wild [36], and Android Control [22]. The data combine element-level supervision from RICO/VINS with action-level interaction semantics from AITW/AC, which we convert into instruction-conditioned GUI grounding samples.

We annotate screenshots with an **80-category** two-level taxonomy (**9 parent classes + 71 child classes**) aligned with common front-end component semantics, drawing on naming and interaction patterns found in libraries such as Ant Design [1], Material UI [30], and Vant [49]. A human-in-the-loop bootstrapping pipeline expands an annotated seed set with a post-trained detection model [19], OCR, and color labels, followed by human refinement; agreement reaches Cohen's $\kappa > 0.82$ on a 500-image held-out set. Each element receives a category and a bounding box; text-like elements additionally receive OCR text and text-color annotations. The taxonomy distinguishes functional roles (e.g., `ReturnButton` versus `CloseButton`), component states, and truncated elements near screen boundaries. The CLIP-deduplicated corpus draws approximately 7.5K screenshots from each source. The complete taxonomy and distribution analyses appear in Appendices A and B.

3.2 Generation Corpus and Rendering Pipeline

For generation, we retain the renderable JSON DSL used by UI-UG [46] instead of raw HTML/CSS, as it offers a compact, structured, and cross-platform interface for progressive UI rendering, following the broader industrial trend of DSL-to-UI systems such as Google/A2UI [12] and Vercel/json-renderer [40]. We improve the corpus with stronger MLLM distillation, richer requirements, and stricter rendering-based filtering. The final generation data contain over **5,000** high-quality requirement–image–JSON tuples; the DSL reduces token usage by **16.7%** on average compared with equivalent HTML. Figure 2 shows the corpus construction pipeline. Each accepted tuple couples a natural-language UI requirement, a reference screenshot, and a JSON specification that passes schema and rendering checks. The shared renderer also executes sampled DSL outputs during generation GRPO.

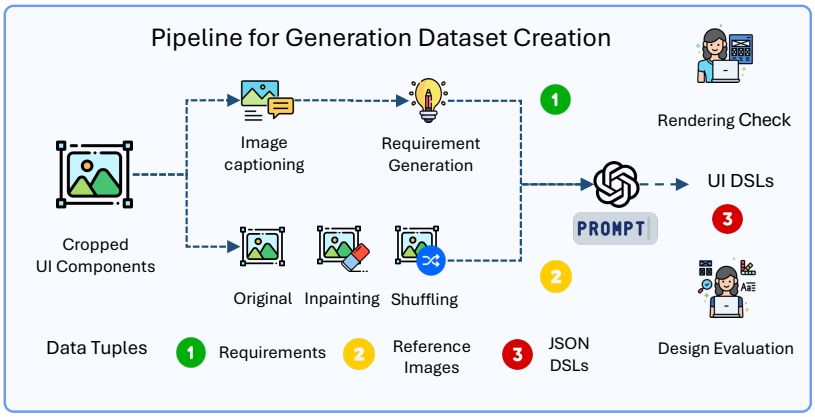


Figure 2: Pipeline for constructing the UI generation dataset. Each tuple consists of a requirement, a reference image, and a corresponding renderable UI JSON DSL.

3.3 Renderable JSON Representation

Our JSON-based DSL describes UI content and styling features. The basic node attributes include:

- `type`: Node type, either `"Component"` or `"Tag"`.
- `name`: If `type` is `"Component"`, the component name from the available components list; if `"Tag"`, an HTML tag name.

- `className`: Tailwind CSS class names using relative layouts. Width/height can be absolute (e.g., `w-[100px]`) or relative (e.g., `w-full`).
- `params`: Content features of the node. Must include `bindType` (`"Static"` or `"Data"`) to indicate whether the value is hardcoded or dynamically bound.
- `children`: A list of child nodes.

The representation separates component identity, layout/styling, content binding, and child structure. It supports progressive rendering as the model produces the structured specification. Figure 10 in Appendix C gives a complete rendered example. A minimal button can be expressed as:

```
{ "type": "Tag", "name": "button",
"className": "rounded-xl bg-blue-600 px-4 py-2",
"params": {"textContent":
{"bindType": "Static", "value": "Continue"}} }
```

4 Method

Given the data and representation in Section 3, we train UI-UG 2 with unified supervised fine-tuning followed by task-specific GRPO. Compared with UI-UG [46], UI-UG 2 uses a smaller 4B backbone and replaces offline generation preference learning with online render-aware GRPO.

4.1 Supervised Fine-tuning

SFT Data Construction. We construct a unified SFT dataset for **referring**, **grounding**, and **generation**, with all targets expressed in JSON for reliable parsing. Referring predicts category, OCR text, and color from a given box; grounding covers dense full-screen localization and instruction-conditioned captioned grounding; generation maps a requirement and optional reference screenshot to a structured JSON DSL. Dense-grounding boxes are sorted by category frequency and top-left coordinates to expose repeated layout regularities.

Model Fine-tuning. We initialize **UI-UG 2-4B-SFT** from Qwen3-VL-4B-Instruct, freeze the ViT encoder, and train the LLM and VL adapter. The training mixture and hyperparameters are reported in Section 4.3.

4.2 Reinforcement Learning via GRPO

While SFT provides a strong initialization, it does not directly optimize task-specific quality metrics. We therefore further optimize UI-UG 2 with GRPO, using rewards tailored to UI understanding and generation.

UI Understanding Rewards.

(1) Referring Reward. The referring reward decomposes element description into category, OCR, and color prediction:

$$R_{\text{ref}} = R_{\text{fmt}} + R_{\text{cls}} + \mathbf{1}[t^* \in \mathcal{T}_{\text{txt}}] \cdot (R_{\text{ocr}} + R_{\text{col}}) \tag{1}$$

where t^* is the target element type, $R_{\text{fmt}} \in \{0, 1\}$ and $R_{\text{cls}} \in \{0, 1\}$ denote format validity and category correctness, and $\mathcal{T}_{\text{txt}} = \{\text{SingleLineText}, \text{MultiLineText}, \text{MoneyLineText}\}$ activates OCR/color rewards only for text elements.

The **OCR reward** uses normalized edit distance (NED) to provide smooth partial credit:

$$R_{\text{ocr}} = 1 - \frac{\text{ED}(\hat{s}, s^*)}{\max(|\hat{s}|, |s^*|)} \tag{2}$$

where $\hat{s}$ and s^* denote predicted and ground-truth strings; if both are empty, $R_{\text{ocr}} = 1$. NED provides smooth partial credit for common UI OCR ambiguities.

The color reward is computed in perceptually uniform CIE L*a*b* space:

$$R_{\text{col}} = \max\left(0,\ 1 - \frac{\Delta E_{ab}(\hat{\mathbf{c}}, \mathbf{c}^*)}{50}\right) \tag{3}$$

where $\hat{\mathbf{c}}$ and $\mathbf{c}^*$ denote the predicted and ground-truth colors, ΔE_{ab} denotes the CIE76 color difference, and 50 normalizes the practical perceptual range.

(2) Grounding Reward. The grounding task contains dense grounding and captioned grounding. For dense full-screen localization, we adopt Dual-IoU [42], which captures both recall-oriented coverage and precision-oriented fidelity:

$$R_{\text{rec}} = \frac{1}{N}\sum_{i=1}^{N}\max_{j:\hat{c}_j=c_i}\text{IoU}(\hat{b}_j, b_i), \quad R_{\text{pre}} = \frac{1}{M}\sum_{j=1}^{M}\max_{i:c_i=\hat{c}_j}\text{IoU}(b_i, \hat{b}_j) \tag{4}$$

$$R_{\text{dense}} = R_{\text{rec}} + R_{\text{pre}} + R_{\text{fmt}} \tag{5}$$

where N and M are the numbers of ground-truth and predicted boxes, respectively, and c_i and $\hat{c}_j$ are their categories. Empty category-matched sets yield 0; the corresponding average is defined as 0 when its box set is empty. This dual formulation prevents degenerate solutions: optimizing recall alone encourages over-detection, whereas optimizing precision alone encourages under-detection.

Captioned grounding localizes a specific element from a natural-language instruction:

$$R_{\text{cap}} = \text{IoU}(\hat{b}, b^*) + R_{\text{fmt}} \tag{6}$$

where $\hat{b}$ and b^* denote the predicted and ground-truth bounding boxes, respectively.

Since dense and captioned grounding are triggered by different prompts, we use a task-conditional grounding reward:

$$R_{\text{ground}} = \mathbf{1}[\text{dense}] \cdot R_{\text{dense}} + \mathbf{1}[\text{cap}] \cdot R_{\text{cap}} \tag{7}$$

UI Generation Reward.

We design a three-part generation reward combining **format validity**, **renderability**, and **perceptual quality**.

(1) Format Reward. $R_{\text{fmt}} \in \{0, 1\}$ checks JSON parseability and conformance to the generation schema. For referring, format checks require category, text, and text-color fields; grounding uses the task's coordinate schema, with category labels required for dense predictions.

(2) Rendering Reward. R_{render} checks whether the DSL can be rendered without execution or layout errors. UI-UG 2 renders each sampled output online with a server-side Playwright pipeline before computing visual rewards, aligning optimization with inference-time artifacts.

(3) Perceptual Quality Reward. We model perceptual quality with **LLM-as-a-Judge** and **Contrastive Preference Ranking (CPR)**, which provide complementary absolute and relative signals. Following Web2Code [50], rendered UIs are scored by Qwen3-VL-235B-A22B along four dimensions: (1) Color & Aesthetic Design, (2) Visual Structure & Alignment, (3) Textual & Content Consistency, and (4) Interface Usability & Interactivity:

$$R_{\text{judge}} = \frac{1}{4}\sum_{k=1}^{4}\frac{s_k}{10} \tag{8}$$

where $s_k \in [0, 10]$ is the judge score on dimension k. This online render-and-judge loop reduces the mismatch between offline preference data and inference-time UI artifacts.

CPR mitigates calibration drift in absolute scoring by adding round-robin pairwise comparisons within each GRPO rollout group. For G sampled responses and $K{=}4$ quality dimensions:

$$R_{\text{cpr}}^{(i)} = \frac{1}{(G-1)\cdot K}\sum_{j\neq i}\sum_{k=1}^{K} w_k(y^{(i)}, y^{(j)}) \tag{9}$$

where $y^{(i)}$ and $y^{(j)}$ are the i-th and j-th sampled responses, and $w_k \in \{0, 0.5, 1\}$ indicates lose/tie/win on dimension k.

The perceptual quality reward combines both signals:

$$R_{\text{perc}} = \alpha \cdot R_{\text{judge}} + (1-\alpha)\cdot R_{\text{cpr}}, \quad \alpha = 0.5 \tag{10}$$

The final generation reward integrates all three components:

$$R_{\text{gen}} = R_{\text{fmt}} + R_{\text{render}} + R_{\text{perc}} \tag{11}$$

where R_{perc} is computed only when both $R_{\text{fmt}} = 1$ and $R_{\text{render}} = 1$.

Sequential GRPO Training. We use sequential task optimization, as supported by the schedule ablation in Section 5.3. The selected curriculum optimizes generation, then referring, then grounding, with one GRPO epoch per stage. Training settings are reported below.

4.3 Training Configuration

We train the policy on 4 A100 80GB GPUs. SFT uses 120K instruction samples (approximately 0.6B tokens) with a referring:dense/GUI-grounding:generation ratio of 3:3:4. GRPO uses 6,000 referring samples, 6,000 grounding samples, and 2,000 generation samples, with eight rollouts per prompt. The policy-training time is approximately two days for SFT and two days for GRPO; it does not include a separate accounting of judge-service compute.

Table 1 summarizes the reported SFT and GRPO settings.

Table 1: Training hyperparameters for SFT and GRPO stages.

Hyperparameter	**SFT**	**GRPO**
Base Model	Qwen3-VL-4B-Instruct	
Frozen Modules	ViT Encoder	
Trainable Modules	LLM + VL Adapter	
Max Learning Rate	1e-5	1e-6
LR Schedule	Warmup-Stable-Decay	Warmup-Stable-Decay
Warmup Ratio	0.05	0.03
Optimizer	AdamW	AdamW
Weight Decay	0.01	0.01
Max Sequence Length	8192	8192
Global Batch Size	16	64
Epochs	2	1 per stage
# SFT Samples	120K ($\sim$0.6B tokens)	—
Task Ratio (Ref:Grd:Gen)	3:3:4	—
# Rollouts per Prompt	—	8
KL Penalty Coefficient	—	0.01
GRPO Samples (Referring)	—	6,000
GRPO Samples (Grounding)	—	6,000
GRPO Samples (Generation)	—	2,000
Hardware	4$\times$ A100 80GB	
Training Time	$\sim$2 days	$\sim$2 days

The hardware above refers to policy-model training. The Qwen3-VL-235B-A22B judge and the Playwright rendering service are deployed separately and queried asynchronously during generation GRPO.

5 Experiments

We compare UI-UG 2-4B against both general-purpose MLLMs (GPT-5.2, Claude Sonnet 4.5, Gemini 3.1 Pro, Kimi K2.5 [38], GLM-4.5V [16], Mistral Large 3 [29], and the Qwen3-VL series) and UI-specific models (Ferret-UI2, OmniParser V2, ShowUI, UI-TARS, GUI-Owl, and UI-Venus) on UI understanding and UI generation. Compared with UI-UG [46], the evaluation here is broader in both task coverage and evidence: we add GUI grounding on ScreenSpot mobile benchmarks, use confidence-free dense-grounding metrics tailored to MLLM outputs, and include broader ablations on online RL, reward transfer, and understanding-generation interaction.

5.1 UI Understanding

UI understanding consists of Referring, Dense Grounding, and GUI Grounding. Referring and Dense Grounding are evaluated on our in-house dataset, while GUI Grounding is evaluated on public

mobile benchmarks. The former assesses the proposed annotation space within the curated corpus; ScreenSpot provides complementary external single-target localization evidence.

Evaluation. We evaluate **Referring** by predicting category, OCR text, and color attribute from a ground-truth box, using Category Accuracy (Cat), OCR-Acc (Eq. 2), and Color-Acc (Eq. 3). **Dense Grounding** is evaluated by F1, Precision, Recall, and mIoU; because MLLMs do not provide confidence scores, we use set-matching F1 rather than AP-style metrics. **GUI Grounding** is evaluated by click accuracy on the mobile splits of ScreenSpot V1 [8] and ScreenSpot-V2 [45], yielding 502 and 449 samples after mobile-only filtering. All MLLMs use standardized prompts with temperature 0 and model-specific coordinate adaptations when needed. For models whose interfaces do not support certain tasks, we mark the corresponding entries as unsupported in Table 2; UI-UG-7B [46] is mapped to our taxonomy for comparison.

Dense set matching. After parsing and discarding malformed or out-of-taxonomy predictions, we match predicted and reference boxes with a Hungarian assignment whose cost is $1 - \mathrm{IoU}$ for equal categories and 1 otherwise. A true positive requires equal categories and IoU ≥ 0.5. Precision, recall, and F1 are computed per image and then averaged; mIoU is computed over category-correct true-positive matches. Consequently, the reported mean F1 need not equal the harmonic mean of mean precision and mean recall. Appendix E provides the step-by-step protocol.

Results. Table 2 shows that UI-UG 2-4B achieves the best overall performance on Referring and Dense Grounding.

On **Referring**, general-purpose MLLMs remain inconsistent: the strongest closed-source baseline, Gemini 3.1 Pro, reaches 0.535 category accuracy, and the best open-source general model, Kimi K2.5, reaches 0.514. Relative to UI-UG-7B, UI-UG 2 pushes the unified setting to a substantially stronger operating point: UI-UG 2-4B-SFT already reaches 0.904/0.930/0.727 on Cat/OCR/Color, and GRPO further improves this to **0.906/0.938/0.739**. The same-backbone SFT-to-GRPO comparison isolates the additional post-training gain. The comparison to UI-UG-7B also changes the backbone, label space, and data, so it does not isolate any single design choice.

On **Dense Grounding**, the margin is larger: the best general-purpose model, Gemini 3.1 Pro, achieves 0.471 F1, while UI-UG 2-4B-SFT reaches 0.742 F1 and 0.865 mIoU, and GRPO improves this further to **0.777 F1** and **0.894 mIoU** ($+0.035$ F1 and $+0.029$ mIoU over SFT). Relative to UI-UG-7B, the new model again advances the unified setting by a wide margin, with the difference most visible in recall because the earlier 20-class label space cannot fully cover our 80-category protocol. Among UI-specific models, ShowUI and UI-TARS expose point/action-based single-target interfaces and do not provide the exhaustive typed boxes or referring attributes required by this protocol, whereas GUI-Owl and UI-Venus-Ground retain grounding ability.

On **GUI Grounding**, UI-UG 2-4B attains 0.902/0.929 on ScreenSpot-V1/V2 mobile, outperforming most open-source general MLLMs and competitive 7B specialists such as UI-TARS-7B (0.886/0.911). Several larger general-purpose models and task-specific baselines remain higher, including UI-Venus-Ground-7B (0.910/0.951) and UI-Venus-Ground-72B (0.922/0.971), likely because they are optimized only for single-target localization. Still, UI-UG 2-4B clearly surpasses its base model Qwen3-VL-4B, validating the effectiveness of the unified training recipe.

Table 2: UI understanding results. Best results are in **bold**; second-best are underlined. −: unknown; †: unsupported by the native output interface. Unsupported entries are excluded from task rankings.

		Referring			Dense Grounding				GUI Grounding	
Model	**Params**	Category	OCR	Color	F1	Precision	Recall	mIoU	SS-V1-Mobile	SS-V2-Mobile
Closed-source										
GPT-5.2 [32]	–	0.415	0.551	0.611	0.048	0.045	0.052	0.656	0.492	0.352
Claude Sonnet 4.5 [2]	–	0.484	0.830	0.628	0.368	0.415	0.330	0.757	0.187	0.232
Gemini 3.1 Pro [13]	–	0.535	0.690	0.501	0.471	0.461	0.482	0.759	0.900	0.951
Open-source										
Qwen3-VL-4B [3]	4B	0.310	0.775	0.417	0.213	0.354	0.152	0.783	0.861	0.886
Qwen3-VL-8B [3]	8B	0.463	0.830	0.614	0.289	0.378	0.234	0.729	0.466	0.918
Qwen3-VL-30B [3]	30B (A3B)	0.407	0.812	0.571	0.221	0.315	0.170	0.775	0.912	0.940
Qwen3-VL-235B [3]	235B (A22B)	0.449	0.855	0.484	0.295	0.385	0.239	0.766	**0.926**	0.958
Kimi K2.5 [38]	1T (A32B)	0.514	0.845	0.689	0.292	0.312	0.274	0.727	0.910	0.953
GLM-4.5V [16]	106B (A12B)	0.352	0.799	0.552	0.264	0.315	0.227	0.742	0.616	0.933
Mistral Large 3 [29]	675B (A41B)	0.321	0.628	0.552	0.179	0.192	0.180	0.723	0.161	0.261
UI-specific										
Ferret-UI2-2B [23]	2B	0.470	0.220	0.312	0.031	0.036	0.027	0.667	0.380	0.479
Ferret-UI2-8B [23]	8B	0.493	0.284	0.312	0.031	0.036	0.027	0.667	0.538	0.521
OmniParser V2 [28]	0.02B	†	†	†	0.342	0.299	0.401	0.715	0.612	0.635
ShowUI-2B [24]	2B	†	†	†	†	†	†	†	0.823	0.853
UI-TARS-7B [33]	7B	†	†	†	†	†	†	†	0.886	0.911
UI-TARS-72B [33]	72B	†	†	†	†	†	†	†	0.890	0.902
GUI-Owl-7B [47]	7B	0.174	0.561	0.390	0.013	0.007	0.044	0.855	0.898	0.931
GUI-Owl-32B [47]	32B	0.176	0.561	0.392	0.014	0.008	0.045	0.855	0.898	0.940
UI-Venus-Ground-7B [14]	7B	0.268	0.821	0.480	0.081	0.128	0.059	0.811	0.910	0.951
UI-Venus-Ground-72B [14]	72B	0.451	0.867	0.627	0.342	0.511	0.257	0.778	0.922	**0.971**
UI-UG-7B [46]	7B	0.738	0.759	0.583	0.447	0.653	0.340	0.840	0.637	0.656
Ours										
UI-UG 2-4B-SFT	4B	0.904	0.930	0.727	0.742	0.798	0.693	0.865	0.869	0.889
UI-UG 2-4B	4B	**0.906**	**0.938**	**0.739**	**0.777**	**0.832**	**0.729**	**0.894**	0.902	0.929

5.2 UI Generation

Evaluation. We evaluate generation by JSON format compliance, render success rate, and four MLLM-scored dimensions: Color & Aesthetic Design, Visual Structure & Alignment, Textual & Content Consistency, and Interface usability & Interactivity. The four reported judge dimensions are normalized to $[0, 1]$. The reported composite adds the two validity rates and the four aggregate judge dimensions:

$$S_{\text{gen}} = p_{\text{JSON}} + p_{\text{render}} + \sum_{k=1}^{4} \bar{q}_k, \qquad S_{\text{gen}} \in [0, 6], \tag{12}$$

where p_{JSON} and p_{render} are the reported JSON-validity and rendering-success rates, and $\bar{q}_k$ is the reported mean normalized score on judge dimension k. This composite sums the four visual dimensions, whereas the training reward in Eq. 11 uses their average together with CPR. Training and evaluation share the quality rubric; the generation score should therefore be interpreted together with format/rendering checks and the human-reference study rather than as an independent measure of all aspects of UI quality.

Human-reference judge validation. On 100 generated UI cases scored independently by three professional UI designers, the Qwen3-VL-235B-A22B judge has a mean absolute deviation of 0.17 and residual variance of 0.046 against the averaged human reference. Gemini 3.1 Pro has the lowest deviation (0.08) in this comparison; Qwen is used for the training reward for deployment and repeated-evaluation practicality. The complete comparison and scoring prompts are in Appendices G.1 and D.2. This post-hoc study does not establish human alignment at every RL update.

Results. Table 3 shows that UI-UG 2-4B achieves the best overall generation quality and improves the average total score by **5.8%** over the SFT model. The comparison demonstrates a stronger compact operating point than UI-UG-7B, while changes in backbone, data, and optimization prevent attributing the full difference to GRPO alone. In particular, UI-UG used offline DPO over a fixed preference dataset, whereas UI-UG 2 uses online render-aware GRPO, aligning optimization with the final executable DSL after actual rendering. This advantage is also reflected in comparisons to strong baselines: UI-UG 2-4B surpasses Claude Sonnet 4.5 (4.70), GPT-5.2 (4.33), and Gemini 3.1 Pro (3.89), and also outperforms Qwen3-VL-235B, which uses roughly **59**× as many total

parameters and **5.5**× as many active parameters. Larger general models also do not reliably produce valid DSL outputs: GLM-4.5V and Qwen3-VL-30B show notably reduced format compliance (0.60 and 0.54), indicating that general instruction tuning alone is insufficient for structured UI generation.

Representation efficiency. The JSON DSL reduces token count by about **16.7%** relative to equivalent HTML in the reported corpus comparison. Appendix H separates the measured UI-UG 2 latency from active-parameter-based estimates for larger models; these estimates are not same-hardware measurements of those models.

Table 3: UI generation results. JSON: format compliance; Rend: render success; Col/Str/Txt/Int: four judge dimensions; Sum: composite score (maximum 6). Best results are in **bold**; second-best are underlined.

Model	JSON	Rend	Col	Str	Txt	Int	Sum
Closed-source							
GPT-5.2 [32]	0.99	0.96	0.58	0.62	0.58	0.60	4.33
Claude Sonnet 4.5 [2]	0.96	0.94	0.69	0.70	0.71	0.70	4.70
Gemini 3.1 Pro [13]	**1.00**	0.97	0.42	0.53	0.48	0.49	3.89
Open-source							
Qwen3-VL-4B [3]	0.98	0.96	0.59	0.59	0.60	0.58	4.30
Qwen3-VL-8B [3]	0.67	0.66	0.61	0.62	0.59	0.61	3.76
Qwen3-VL-30B [3]	0.54	0.48	0.64	0.63	0.55	0.62	3.46
Qwen3-VL-235B [3]	**1.00**	**0.98**	0.66	0.68	0.61	0.66	4.59
Kimi K2.5 [38]	0.86	0.86	0.66	0.68	0.61	0.65	4.32
GLM-4.5V [16]	0.60	0.59	0.64	0.66	0.62	0.64	3.75
Mistral Large 3 [29]	0.99	0.95	0.56	0.54	0.55	0.54	4.13
UI-specific							
UI-UG-7B [46]	0.93	0.89	0.62	0.57	0.63	0.56	4.20
Ours							
UI-UG 2-4B-SFT	0.93	0.93	0.71	0.67	0.74	0.67	4.65
UI-UG 2-4B	0.99	**0.98**	**0.73**	**0.72**	**0.79**	**0.71**	**4.92**

5.3 Ablation Study

RL Training Strategy. We ablate RL objectives and training schedules. Figure 3 reports improvement over the SFT baseline in referring accuracy (ΔRef Acc), grounding F1 (ΔGrd F1), and generation score (ΔGen) across joint training and 1/2/3-stage sequential RL configurations.

Finding 1: RL improves over SFT. Task-aligned RL consistently improves each target capability over the SFT baseline: single-stage GenerationRL raises generation score by +4.43%, GroundingRL improves grounding F1 by +4.02%, and ReferringRL increases referring accuracy by +1.47%. Notably, understanding-oriented RL also benefits generation, suggesting that stronger element-level understanding transfers to UI generation. The gains further scale with the number of training stages, as 2-stage and 3-stage sequential configurations progressively improve all three metrics.

Finding 2: Sequential outperforms joint optimization. Joint RL yields Gen +2.73%, Grd F1 +2.70%, and Ref Acc +0.86%, below the selected sequential configurations at the same reported training budget. The tasks differ in response length and reward distribution; this pattern is consistent with greater interference during simultaneous optimization. The pattern is consistent with prior work showing that heterogeneous rewards are difficult to combine into a single stable objective [44].

5.3.1 Dense Grounding Reward Components

The dense grounding reward $R_{\text{dense}} = R_{\text{fmt}} + R_{\text{pre}} + R_{\text{rec}}$ comprises three components. We compare combinations of these terms: (1) a format reward R_{fmt} enforcing valid JSON with parseable bounding boxes; (2) a precision-oriented term $R_{\text{pre}} = \frac{1}{M}\sum_{j=1}^{M} \max_i \text{IoU}(b_i, \hat{b}_j)$ to suppress spurious predictions; and (3) a recall-oriented term $R_{\text{rec}} = \frac{1}{N}\sum_{i=1}^{N} \max_j \text{IoU}(\hat{b}_j, b_i)$ to improve coverage. Table 4 reports an additive ablation from the SFT baseline.

From Table 4, adding R_{fmt} first gives a small but consistent improvement over SFT (F1: 0.742 → 0.745; mIoU: 0.865 → 0.868), mainly by reducing invalid outputs. Adding R_{pre} on top of R_{fmt} further improves overall quality (Precision: 0.836; F1: 0.763; mIoU: 0.877), showing that explicit precision control effectively suppresses noisy boxes.

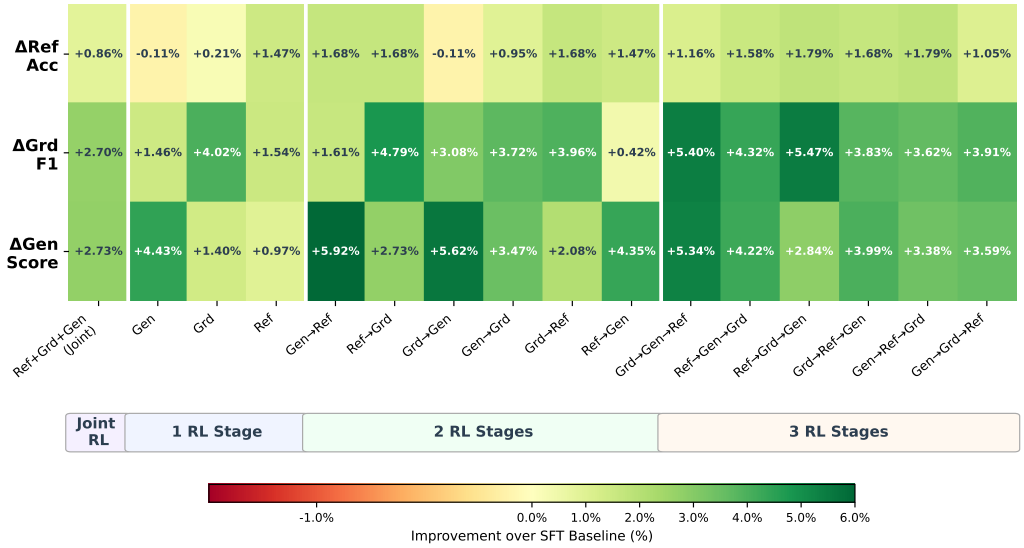


Figure 3: RL strategy ablation. Each cell shows improvement over SFT; sequential multi-stage training yields larger gains.

Table 4: Ablation of dense grounding reward components. Each row specifies the reward combination applied to the SFT baseline. Precision, Recall, and F1 are averaged over samples; therefore, the reported F1 is not necessarily the harmonic mean of the averaged Precision and Recall.

Configuration	Precision	Recall	F1	mIoU
SFT (no RL)	0.798	0.693	0.742	0.865
$+R_{\text{fmt}}$	0.803	0.697	0.745	0.868
$+R_{\text{fmt}} + R_{\text{pre}}$	**0.836**	0.712	0.763	0.877
$+R_{\text{fmt}} + R_{\text{rec}}$	0.754	**0.742**	0.727	0.836
$+R_{\text{fmt}} + R_{\text{pre}} + R_{\text{rec}}$	0.832	0.729	**0.777**	**0.894**

An important phenomenon appears when training with recall emphasis alone: the $R_{\text{fmt}} + R_{\text{rec}}$ setting increases Recall to 0.742, but Precision drops noticeably to 0.754, and both F1 (0.727) and mIoU (0.836) become worse than the SFT baseline. This indicates a reward-hacking pattern, where the model tends to output more boxes to gain recall reward. Therefore, bidirectional IoU supervision is necessary. The full objective $R_{\text{fmt}} + R_{\text{pre}} + R_{\text{rec}}$ (UI-UG 2-4B) restores the balance and achieves the best final performance (F1: 0.777, mIoU: 0.894), with gains over SFT of +0.035 F1 and +0.029 mIoU.

5.4 Understanding–Generation Transfer Analysis

We analyze the interaction between UI understanding and generation from three perspectives.

Experiment 1: Effect of Understanding-Generation Data Mixing Ratio during SFT. We vary the proportion of understanding data (referring + grounding) and generation data during SFT under a fixed SFT budget of 120K samples (approximately 0.6B tokens).

Figure 4 shows a non-monotonic pattern. Uxx denotes that $xx\%$ of training data comes from understanding tasks. Generation peaks at **U20 (4.760)**, improving over the generation-only setting U0 (4.668). Because the total training budget is fixed, this gain comes from complementary supervision rather than larger scale. Beyond U20, generation quality gradually declines as the model receives less generation-specific supervision. In contrast, understanding improves monotonically with understanding ratio: referring accuracy rises from 0.789 (U10) to 0.838 (U100), and grounding F1 from 0.475 to 0.690. This supports an asymmetric transfer pattern in this fixed-budget mixture: moder-

ate understanding supervision benefits generation, while replacing understanding supervision with generation does not improve the measured understanding metrics.

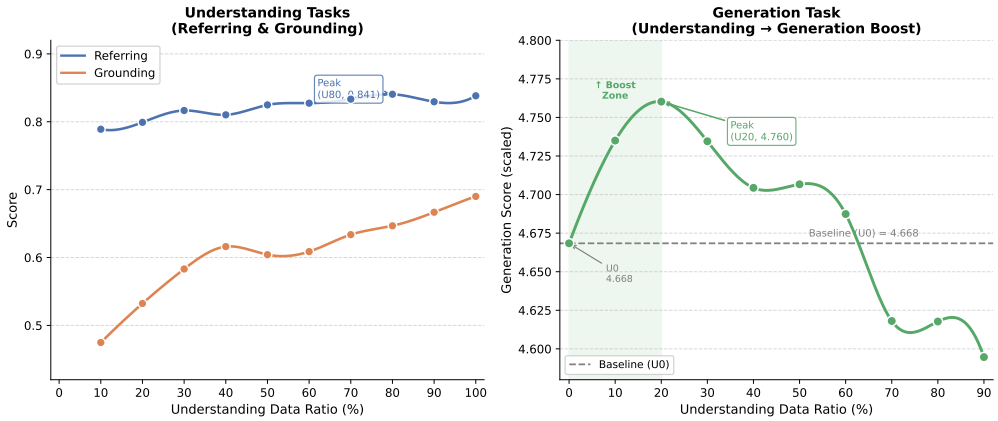


Figure 4: Effect of understanding-data ratio during SFT. Generation peaks at U20, while understanding metrics increase monotonically.

Experiment 2: Dimension-Specific Understanding Rewards for Generation. We test whether different understanding rewards improve their semantically aligned generation dimensions. Each reward is applied independently during RL, and the resulting change in generation score is measured relative to the SFT baseline.

Table 5 shows a clear alignment between understanding skills and generation dimensions. $+R_{\text{col}}$ gives the notable gain on Color & Aesthetic Design (+1.60%), $+R_{\text{ocr}}$ gives the largest gain on Textual & Content Consistency (+2.48%), and $+R_{\text{dense}}$ produces the largest overall benefit, especially on Visual Structure & Alignment (+4.72%). In contrast, $+R_{\text{cap}}$ yields only marginal changes, suggesting that single-element GUI grounding transfers less effectively to holistic generation. The transfer is selective rather than uniform: several off-dimension changes are negative, and GUI-grounding reward alone gives little benefit. The combined model's improvement does not imply that each understanding objective improves every generation dimension.

Table 5: Relative change in generation scores after applying each understanding reward independently in RL training. Notable Gain is shown in **bold**(>1%).

Reward	ΔCol	ΔStr	ΔTxt	ΔInt
$+R_{\text{cls}}$ (Category)	**+1.31%**	+0.86%	**+1.31%**	−1.49%
$+R_{\text{col}}$ (Color)	**+1.60%**	−0.39%	−0.18%	+0.41%
$+R_{\text{ocr}}$ (Text)	−0.11%	−1.45%	**+2.48%**	−0.13%
$+R_{\text{dense}}$ (Dense Gnd.)	**+1.64%**	**+4.72%**	+0.25%	**+1.74%**
$+R_{\text{cap}}$ (GUI Gnd.)	+0.20%	−0.27%	−0.25%	−0.17%

Experiment 3: Cross-Task Attention Similarity Analysis. To probe the mechanism behind the understanding–generation synergy, we compare text-to-vision attention patterns across tasks. For sample b at layer l, we aggregate text-to-vision attention over heads, extract the visual-token slice, and apply min–max normalization to obtain a heatmap $\hat{\mathbf{V}}_{b,l} \in \mathbb{R}^{H\times W}$, whose flattened form is $\hat{\mathbf{v}}_{b,l}$. Averaging $\hat{\mathbf{v}}_{b,l}$ over samples within each task gives the layer-wise task representation $\hat{\mathbf{v}}_l^{\,p}$. We then define task similarity as the mean cosine similarity over the L layers:

$$\text{Sim}_{\cos}(p,q) = \frac{1}{L}\sum_{l=1}^{L} \frac{\hat{\mathbf{v}}_l^{\,p}\cdot\hat{\mathbf{v}}_l^{\,q}}{\|\hat{\mathbf{v}}_l^{\,p}\|_2\,\|\hat{\mathbf{v}}_l^{\,q}\|_2}. \tag{13}$$

Figure 5 shows high similarity across understanding–generation pairs, with generation and dense grounding exhibiting the strongest alignment. This is consistent with the reward-transfer results, but attention similarity is correlational evidence: it does not by itself establish the causal mechanism of transfer.

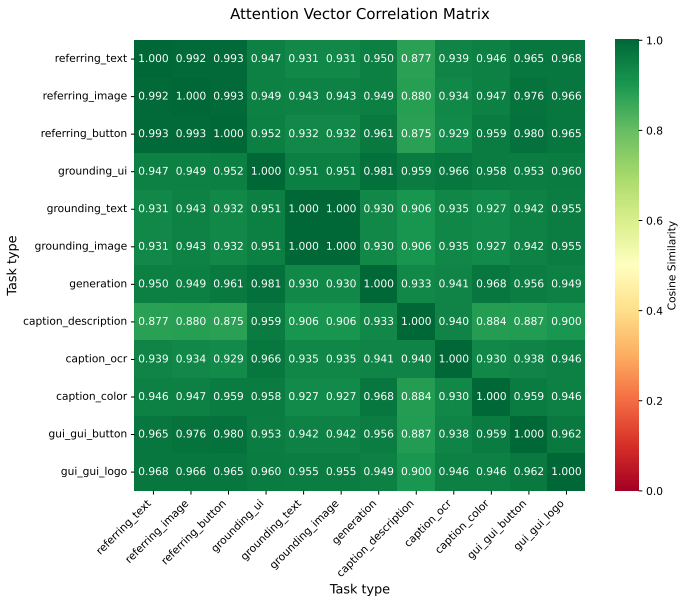


Figure 5: Cosine similarity of text-to-vision attention patterns across understanding and generation tasks. Generation and dense grounding show the strongest alignment.

5.5 Qualitative Generation Results

Figure 6 compares generated interfaces under the same requirements: an event-poster card and a mobile login page. UI-UG 2-4B produces more consistent typography, spacing, component geometry, and global style across these two cases. The examples illustrate the layout and content differences reflected in the aggregate scores; they are qualitative cases rather than a substitute for broader functional testing.

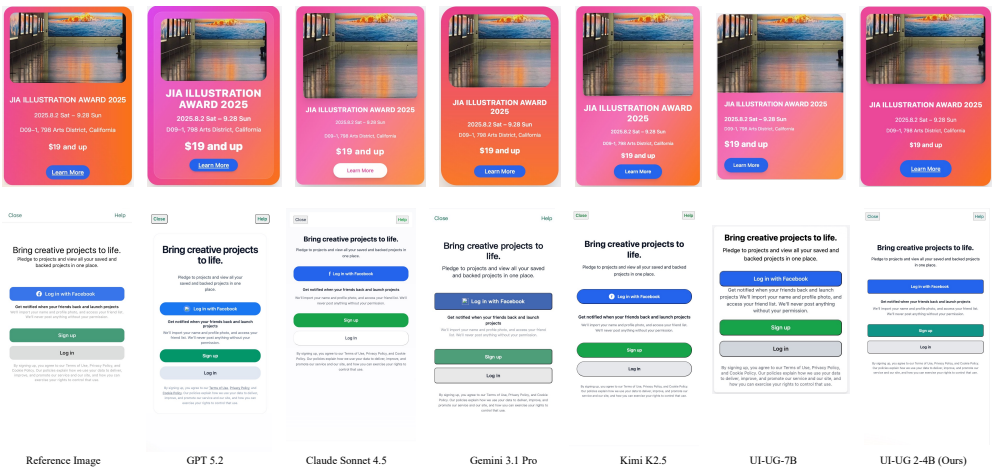


Figure 6: Visual comparison of UI generation quality on two representative cases (poster card and login interface). Each column corresponds to one model under the same prompt condition. UI-UG 2-4B (rightmost) shows better layout stability, hierarchy consistency, and style-control fidelity than representative MLLMs.

6 Discussion and Limitations

Benchmark scope and comparisons. The largest understanding gains are measured on the proposed 80-category corpus and should be interpreted as in-domain results. ScreenSpot mobile evaluates external single-target localization, where several larger or specialized models remain stronger. Native point/action-based specialists do not expose the typed exhaustive predictions required by our dense/referring protocol; unsupported entries indicate an interface mismatch, not a measured failure on their native task. The UI-UG-7B comparison changes the backbone, data, and taxonomy simultaneously, whereas the Qwen3-VL-4B and UI-UG 2-4B-SFT rows provide same-backbone controls.

Generation evaluation and reward alignment. Generation rewards and reported scores share four visual-quality dimensions, so optimization against the rubric may overstate improvements outside that rubric. The human-reference study is limited to 100 cases and is post-hoc rather than a continuous audit during GRPO. Screenshot-based judgments of usability assess visual affordances; they do not replace interactive functional testing, accessibility evaluation, or end-user studies. Online rendering and large-model judging also add service compute beyond the reported policy-training budget.

Transfer and deployment. The fixed-budget mixture and reward ablations support a benefit from selected understanding signals, but some isolated rewards reduce other generation dimensions. Attention alignment is descriptive evidence rather than causal identification. Rare UI types outside the taxonomy, distribution shifts, and environments beyond the studied mobile setting remain open challenges. Finally, the appendix's latency extrapolations use active parameter count as a rough scaling assumption; they are not direct deployment benchmarks of every model.

7 Conclusion

We presented **UI-UG 2**, a unified 4B MLLM for mobile UI understanding and generation. With a front-end-aligned taxonomy, broader grounding supervision, a compact renderable DSL, and online render-aware GRPO, UI-UG 2 achieves strong results across referring, dense grounding, GUI grounding, and UI generation. Our ablations further show asymmetric transfer from fine-grained understanding to generation, suggesting that component-level UI perception is a useful foundation for structured UI synthesis.

References

[1] Ant Group. Ant design. https://ant.design, 2026. Official website, accessed: 2026-01-01.

[2] Anthropic. Introducing claude sonnet 4.5. https://www.anthropic.com/news/claude-sonnet-4-5, 2025. Accessed: 2025-09-25.

[3] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[4] Tony Beltramelli. pix2code: Generating code from a graphical user interface screenshot. In *Proceedings of the ACM SIGCHI symposium on engineering interactive computing systems*, pages 1–6, 2018.

[5] Sara Bunian, Kai Li, Chaima Jemmali, Casper Harteveld, Yun Fu, and Magy Seif Seif El-Nasr. Vins: Visual search for mobile user interface design. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–14, 2021.

[6] Deng Cai, Shipeng Yu, Ji-Rong Wen, and Wei-Ying Ma. Vips: a vision-based page segmentation algorithm, 2003.

[7] Xingyu Chen, Zihan Zhao, Lu Chen, Danyang Zhang, Jiabao Ji, Ao Luo, Yuxuan Xiong, and Kai Yu. Websrc: A dataset for web-based structural reading comprehension. *arXiv preprint arXiv:2101.09465*, 2021.

[8] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. Seeclick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, 2024.

[9] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

[10] Biplab Deka, Zifeng Huang, and Ranjitha Kumar. Erica: Interaction mining mobile apps. In *Proceedings of the 29th annual symposium on user interface software and technology*, pages 767–776, 2016.

[11] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th annual ACM symposium on user interface software and technology*, pages 845–854, 2017.

[12] Google. A2ui. https://developers.googleblog.com/introducing-a2ui-an-open-project-for-agent-driven-interfaces, 2025. GitHub repository, accessed: 2025-12.

[13] Google. Gemini 3.1 pro: A smarter model for your most complex tasks. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro, 2026. Accessed: 2026-02-19.

[14] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025.

[15] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[16] Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, et al. Glm-4.5 v and glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv preprint arXiv:2507.01006*, 2025.

[17] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[18] Vanita Jain, Piyush Agrawal, Subham Banga, Rishabh Kapoor, and Shashwat Gulyani. Sketch2code: transformation of sketches to ui in real-time using deep neural network. *arXiv preprint arXiv:1910.08930*, 2019.

[19] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8, 2023. URL https://github.com/ultralytics/ultralytics.

[20] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2struct: Screenshot parsing as pretraining for visual language understanding. In *International Conference on Machine Learning*, pages 18893–18912. PMLR, 2023.

[21] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 8778–8786, 2025.

[22] Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems*, 37:92130–92154, 2024.

[23] Zhangheng Li, Keen You, Haotian Zhang, Di Feng, Harsh Agrawal, Xiujun Li, Mohana Prasad Sathya Moorthy, Jeff Nichols, Yinfei Yang, and Zhe Gan. Ferret-ui 2: Mastering universal user interface understanding across platforms. *arXiv preprint arXiv:2410.18967*, 2024.

[24] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19498–19508, 2025.

[25] Xiaodong Liu, Pengcheng He, Weizhu Chen, and Jianfeng Gao. Multi-task deep neural networks for natural language understanding. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4487–4496, 2019.

[26] Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. Visual-rft: Visual reinforcement fine-tuning. *arXiv preprint arXiv:2503.01785*, 2025.

[27] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for pure vision based GUI agent. *arXiv preprint arXiv:2408.00203*, 2024. URL https://arxiv.org/abs/2408.00203.

[28] Yadong Lu, Thomas Dhome-Casanova, Jianwei Yang, and Ahmed Awadallah. OmniParser V2: Turning any LLM into a computer use agent. https://www.microsoft.com/en-us/research/articles/omniparser-v2-turning-any-llm-into-a-computer-use-agent/, 2025. Microsoft Research, February 12.

[29] Mistral AI. Introducing mistral 3. https://mistral.ai/news/mistral-3, 2025. Accessed: 2025-12-02.

[30] MUI. Material ui. https://mui.com/material-ui, 2026. Official website, accessed: 2026-01-01.

[31] Tuan Anh Nguyen and Christoph Csallner. Reverse engineering mobile application user interfaces with remaui (t). In *2015 30th IEEE/ACM international conference on automated software engineering (ASE)*, pages 248–259. IEEE, 2015.

[32] OpenAI. Gpt-5.2 system card. https://openai.com/index/gpt-5-system-card-update-gpt-5-2, 2025. Accessed: 2025-12-11.

[33] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

[34] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.

[35] Abinav Rao and Sujan Rachuri. Do understanding and generation fight? a diagnostic study of dpo for unified multimodal models. *arXiv preprint arXiv:2603.17044*, 2026.

[36] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36: 59708–59728, 2023.

[37] Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, et al. Vlm-r1: A stable and generalizable r1-style large vision-language model. *arXiv preprint arXiv:2504.07615*, 2025.

[38] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

[39] Simon Vandenhende, Stamatios Georgoulis, Wouter Van Gansbeke, Marc Proesmans, Dengxin Dai, and Luc Van Gool. Multi-task learning for dense prediction tasks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(7):3614–3633, 2021.

[40] Vercel. json-renderer. https://github.com/vercel-labs/json-render, 2026. GitHub repository, accessed: 2026–01.

[41] Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo, and Michael R Lyu. Automatically generating ui code from screenshot: A divide-and-conquer-based approach. *arXiv preprint arXiv:2406.16386*, 2024.

[42] Haochen Wang, Xiangtai Li, Zilong Huang, Anran Wang, Jiacong Wang, Tao Zhang, Jiani Zheng, Sule Bai, Zijian Kang, Jiashi Feng, et al. Traceable evidence enhanced visual grounded reasoning: Evaluation and methodology. *arXiv preprint arXiv:2507.07999*, 2025.

[43] Jason Wu, Xiaoyi Zhang, Jeff Nichols, and Jeffrey P Bigham. Screen parsing: Towards reverse engineering of ui models from screenshots. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, pages 470–483, 2021.

[44] Runzhe Wu, Ankur Samanta, Ayush Jain, Scott Fujimoto, Jeongyeol Kwon, Ben Kretzu, Youliang Yu, Kaveh Hassani, Boris Vidolov, and Yonathan Efroni. Imbalanced gradients in rl post-training of multi-

task llms. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3137–3150, 2026.

[45] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. OS-ATLAS: A foundation action model for generalist GUI agents. *arXiv preprint arXiv:2410.23218*, 2024. URL https://github.com/OS-Copilot/OS-Atlas.

[46] Hao Yang, Weijie Qiu, Ru Zhang, Zhou Fang, Ruichao Mao, Xiaoyu Lin, Maji Huang, Zhaosong Huang, Teng Guo, Shuoyang Liu, and Hai Rao. Ui-ug: A unified mllm for ui understanding and generation, 2025. URL https://arxiv.org/abs/2509.24361.

[47] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025.

[48] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-ui: Grounded mobile ui understanding with multimodal llms. In *European Conference on Computer Vision*, pages 240–255. Springer, 2024.

[49] Youzan. Vant. https://vant-ui.github.io/vant/, 2026. Official website, accessed: 2026-01-01.

[50] Sukmin Yun, Rusiru Thushara, Mohammad Bhat, Yongxin Wang, Mingkai Deng, Jinhong Wang, Tianhua Tao, Junbo Li, Haonan Li, Preslav Nakov, et al. Web2code: A large-scale webpage-to-code dataset and evaluation framework for multimodal llms. *Advances in neural information processing systems*, 37: 112134–112157, 2024.

[51] Jingyi Zhang, Jiaxing Huang, Huanjin Yao, Shunyu Liu, Xikun Zhang, Shijian Lu, and Dacheng Tao. R1-vl: Learning to reason with multimodal large language models via step-wise group relative policy optimization. *arXiv preprint arXiv:2503.12937*, 2025.

[52] Ting Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen, and Haoyu Wang. Bridging design and development with automated declarative ui code generation, 2024.

A Complete UI Element Taxonomy

This section provides a full taxonomy reference used throughout our UI understanding and generation pipeline. We include the complete parent-child structure together with front-end correspondence for reproducibility.

Our taxonomy follows the structure in "Mobile UI Component Taxonomy: Category Definitions and Library Mappings," with **80 categories** in total, organized as **9 parent classes + 71 fine-grained subcategories**. The parent classes are annotation super-categories designed for stable dense labeling rather than one-to-one modules in any single front-end library. In this appendix, Ant Design Mobile is used as a practical reference for front-end correspondence. Table 7 further situates our taxonomy against representative public UI datasets.

Compared with coarse UI labels, this taxonomy explicitly models three properties that are critical for UI understanding: (1) **state-level variants** (e.g., checked/unchecked, fold/unfold), (2) **functional variants** (e.g., different button and card roles), and (3) **boundary truncation variants** for partially visible elements near screen edges. These design choices improve semantic coverage and annotation usability for dense grounding and referring tasks. Table 6 provides the complete 9-parent hierarchy and mapping rationale, with subcategory counts summing strictly to 71.

Table 6: Complete UI taxonomy summary (80 classes = 9 parents + 71 subcategories). AntD Mobile is used here as the reference for front-end component correspondence.

Parent Class	Subcategories (complete list)	AntD Correspondence
Card (9)	InfoCard, CommodityCard, MenuCard, InfoCardTruncated, CommodityCardTruncated, Banner, BannerTruncated, NoticeBar, Swiper	AntD Card (functional/truncation variants); AntD NoticeBar; AntD Swiper; custom Banner pattern
Collapse (9)	CollapseFold, CollapseUnFold, Popup, FloatingPanel, List, ListItem, Table, Form, FormItem	AntD Collapse (state-level split); AntD Popup/FloatingPanel; AntD List/List.Item; AntD Form/Form.Item; custom Table
Chart (7)	LineChart, HorizontalBarChart, VerticalBarChart, CircleChart, RadarChart, DashBoardChart, OtherChart	Not in AntD; referenced from AntV/F2, ECharts, and Recharts; custom OtherChart for uncommon types
Navigation (7)	Tabs, NavBar, TabBar, SideBar, IndexBar, TabsItemSelected, TabsItemUnselected	AntD Tabs/NavBar/TabBar/SideBar/IndexBar; custom item-state split (Selected/Unselected)
Progress (6)	ProgressBar, ProgressCircle, Selector, Steps, StepsItem, Scroll	AntD ProgressBar/ProgressCircle; AntD Selector; AntD Steps/Steps.Step; custom Scroll indicator
Button (10)	CommonButton, SideButton, CustomButton, TextButton, CloseButton, MoreButton, HomeButton, ReturnButton, Switch, Dropdown	AntD Button (functional variants); AntD Switch; AntD Dropdown
Text (16)	SingleLineText, MultiLineText, MoneyLineText, Popover, FloatingBubble, Icon, Avatar, Tag, Badge, Stepper, Input, TextArea, SearchBar, Slider, Calendar, Keyboard	AntD Ellipsis (typography); domain-specific MoneyLineText; AntD Popover, FloatingBubble, Avatar, Tag, Badge, Stepper; AntD Input, TextArea, SearchBar, Slider; AntD CalendarPicker, NumberKeyboard
Image (3)	Image, BackgroundImage, QRCode	AntD Image; custom BackgroundImage pattern; AntD QRCode (desktop, via qrcode.react)
Checkbox (4)	CheckboxChecked, CheckboxUnchecked, Radio, Rate	AntD Checkbox (state-level split); AntD Radio; AntD Rate

Table 7: UI element taxonomy coverage comparison. FW-Aligned indicates whether the labels are aligned with front-end component libraries such as Ant Design, Material UI, and Vant.

Dataset	Categories	OCR / Color	Level	FW-Aligned	# Images
RICO [11]	24	× / ×	Element	×	∼72k
VINS [5]	11	× / ×	Element	×	∼4k
AITW [36]	—	× / ×	Action	✓	∼715k
AC [22]	—	× / ×	Action	×	∼1.5M
Ours	**80 (9 + 71)**	✓	Element+Action	✓	**>30k**

B UI Image Dataset Statistics

This section summarizes how the UI understanding dataset is composed and why its distribution is suitable for robust training and evaluation.

B.1 Data Sources and Scale

Following the data setup in the main text (Data Construction section), our curated **UI understanding dataset** is built from four sources: RICO, VINS, Android in the Wild, and Android Control. After CLIP-based deduplication, the merged corpus contains **over 30,000 unique screenshots**, with an almost balanced contribution of roughly ∼7.5k images per source. All samples are annotated under our **80-category taxonomy** (9 parent classes + 71 child classes), providing unified supervision for UI understanding across heterogeneous mobile UI domains.

B.2 Annotation Visualization

To illustrate annotation quality, Figure 7 shows representative visualization results from the four data sources (RICO, VINS, AITW, and AC). Each sample overlays **element category** and **bounding-box location** annotations, demonstrating consistent fine-grained type and position labeling across diverse UI styles.

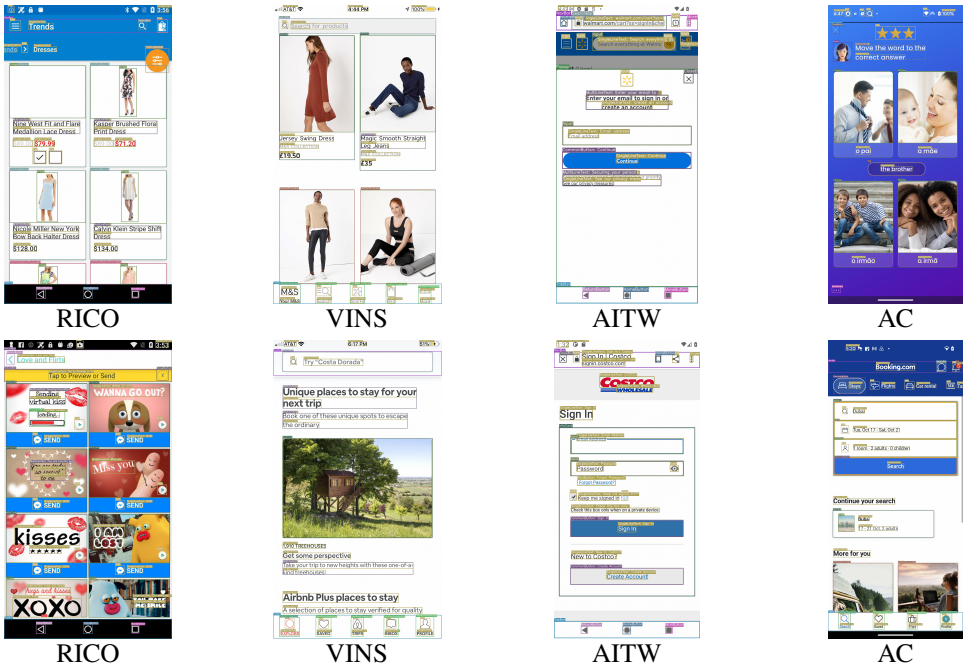


Figure 7: Visualization examples of dataset annotations (2×4). The order in each row is RICO, VINS, AITW, and AC. Overlaid annotations illustrate both UI element categories and bounding-box locations.

B.3 Distribution Balance Analysis

Our final dataset is constructed by **mixing screenshots from all four sources** (RICO, VINS, AITW, and AC), rather than relying on any single dataset. We further analyze the merged data distribution from two complementary perspectives: **scene category composition** and **sample complexity**. Figure 8 shows that the four sources have clearly different scene priors (e.g., AITW is dominated by social-media-like pages, while VINS contains a large proportion of onboarding/guide pages). This heterogeneity is precisely why multi-source mixing is beneficial: it broadens semantic coverage and reduces the risk that the model overfits to one dominant app scenario.

Figure 9 presents complexity-level statistics (Simple/Medium/High) across datasets. The single-source distributions are imbalanced: some datasets are skewed toward simpler layouts, while others retain a broader medium/high tail. After mixing, the resulting corpus better covers the full complexity range, from clean low-density screens to dense and cluttered interfaces. In practice, this helps mitigate the complexity bias of individual datasets and provides more stable supervision for both easy cases (format and basic localization consistency) and hard cases (fine-grained category discrimination and dense grounding under crowded layouts).

In summary, the four-dataset mixture serves as a practical balancing strategy: it compensates for **scene-type imbalance** and **complexity imbalance** in any single source, yielding a more diverse and better-conditioned training distribution for unified UI understanding and generation.

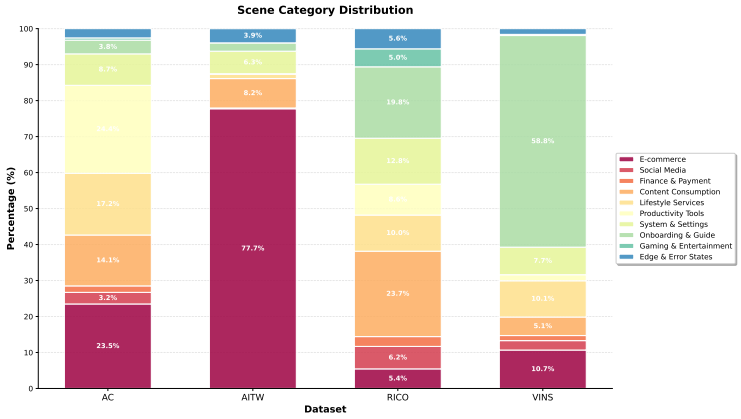


Figure 8: Scene category distribution across AC, AITW, RICO, and VINS. The four sources exhibit complementary scene biases that improve overall diversity.

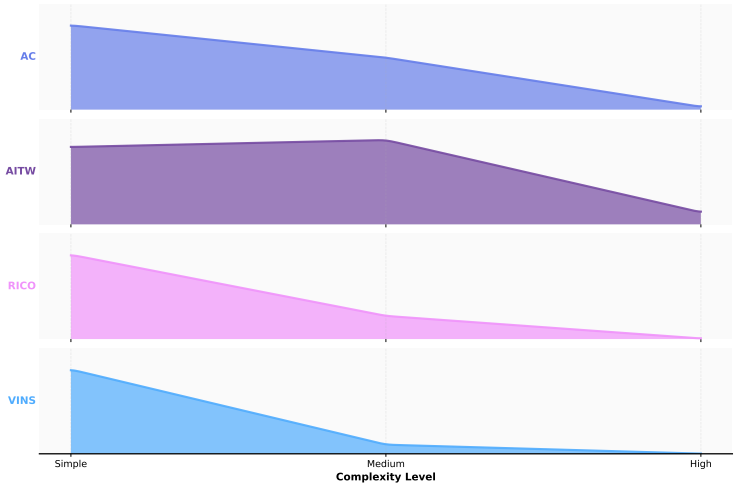


Figure 9: Complexity-level distribution (Simple/Medium/High) for each source dataset. Most samples are simple/medium, with source-dependent medium/high tails.

C Additional JSON DSL Examples

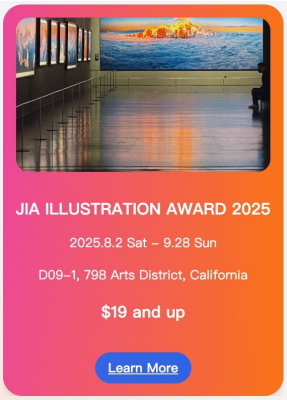
The corpus construction pipeline and node schema are given in Section 3. This appendix provides a rendered dataset example and a longer textual specification.

C.1 DSL Examples

Simple example. A minimal single-button node:

```
{ "type": "Tag", "name": "a",
"className": "inline-flex items-center justify-center font-medium
bg-blue-600 text-white rounded-3xl py-2 px-4",
"params": { "textContent": { "bindType": "Static", "value":
"Learn More" } }
}
```

Complex example from dataset. Figure 10 presents a real example from our generation dataset. The left panel shows the rendered UI card, and the right panel shows the corresponding DSL (truncated for readability). The full DSL comprises a root `div` with nested children including an image container, text hierarchy, and an action button, totaling 499 tokens (counted with OpenAI's tokenizer).



{"type": "Tag", "name": "div", "className": "flex flex-col items-center justify-center bg-gradient-to-r from-pink-500 to-orange-500 rounded-3xl shadow-lg w-full max-w-sm p-4", "children": [{"type": "Tag", "name": "img", "className": "w-full h-48 object-cover rounded-2xl mb-4", "params": {"src": {"bindType": "Static", "value": "mockImageUrl"}}}, {"type": "Tag", "name": "div", "className": "text-center", "children": [{"type": "Tag", "name": "h2", "className": "text-xl font-bold text-white mb-2", "params": {"textContent": {"bindType": "Data", "bindField": "title"}}}, {"type": "Tag", "name": "p", "className": "text-base text-white mb-2", "params": {"textContent": {"bindType": "Data", "bindField": "eventDescription"}}}, {"type": "Tag", "name": "p", "className": "text-base text-white mb-2", "params": {"textContent": {"bindType": "Data", "bindField": "dateRange"}}}, {"type": "Tag", "name": "p", "className": "text-base text-white mb-2", "params": {"textContent": {"bindType": "Data", "bindField": "location"}}}, {"type": "Tag", "name": "p", "className": "text-xl font-bold text-white", "params": {"textContent": {"bindType": "Data", "bindField": "price"}}}]}, {"type": "Tag", "name": "a", "className": "mt-4 inline-flex items-center justify-center font-medium transition-colors focus: outline-none focus: ring-2 focus: ring-offset-2 disabled: opacity-50 disabled: pointer-events-none bg-blue-600 text-white hover: bg-blue-700 active: bg-blue-800 rounded-3xl py-2 px-4 border-0", "params": {"href": {"bindType": "Static", "value": "#"}, "textContent": {"bindType": "Static", "value": "Learn More"}

Figure 10: A real example from our generation dataset. Left: rendered UI card. Right: corresponding JSON DSL (truncated). The full DSL contains nested layout, dynamic data bindings, and Tailwind styling tokens.

D Prompt Templates

For completeness, we list the prompt templates used in SFT, evaluation, and reward-related judging.

D.1 SFT Task Prompts

We provide the prompt templates used during supervised fine-tuning for each task.

Referring prompts.

- "Describe the region {box}."
- "Describe the categories and properties of the UI in area {box}."
- "List the types and characteristics of UI elements in region {box}."
- "Provide details on the UI types and their attributes within area {box}."
- "Explain the categories and features of the UI components in region {box}."
- "Summarize the types and properties of UI present in area {box}."

Dense grounding prompts.

- "Detect all UI elements in the image and return their categories and bounding boxes in JSON format."
- "List all the {category} items."
- "Please list all the {category} in the image."
- "Identify every {category} present in this picture."
- "List all {category} shown in the image."
- "Enumerate each {category} in the provided photo."
- "What are all the {category} in this image? Please enumerate."
- "Can you list every {category} appearing in this image?"
- "Give a detailed list of every {category} in the picture."

GUI grounding prompts.

- "Output the bounding box corresponding to the following instruction: {instruction}. The output should just be the coordinates of a box, in the format [x0, y0, x1, y1]. If the task is infeasible, output [-1, -1, -1, -1]."

Generation prompts.

- "Design a UI card to showcase the details of {ui_description}."
- "Design a concise, clear, and visually appealing UI card to display {ui_description}."

- "Please design a UI card displaying {ui_description}."
- "Please generate a UI card to guide users to the {ui_description}."
- "Design a UI card to display {ui_description}. The following is the mock data for the UI card. {mock_data}."

D.2 LLM-as-a-Judge Scoring Prompt

The following prompt is used for absolute quality scoring in our generation reward:

> You are an advanced AI model equipped with OCR and image understanding capabilities, capable of analyzing visual elements in detail.
>
> Your task is to assess two UI images and output a score between 0 and 10 for each of the following questions.
>
> If the answer to a question is a definite YES, output a score of 10, signifying perfect similarity. Conversely, a definite NO should yield a score of 0, indicating no similarity.
>
> For answers that fall in between, assign a score accordingly, where a higher number indicates a greater degree of similarity. Only provide the numerical score for each question, without any additional text.
>
> Example contexts are provided for clarity. Examples provide the idea, but you can output any number in 0–10 range accordingly. Only output a comma separated list containing 4 numbers. DO NOT give score of 10 for any category unless otherwise the two images are identical.
>
> 1. **Color & Aesthetic Design** (Score: 0–10): How closely do the color schemes of the two UI images align in terms of background and text colors? Evaluate the similarity in hues, saturation, and overall color aesthetics. Is the overall aesthetic appeal (modern, minimalistic, traditional, etc.) similar on both UI images? (e.g., A score of 10 for perfectly matching color schemes and identical aesthetics, including identical hues and saturation levels, 6 for similar color palettes and styles with minor variations, and 0 for starkly different color schemes and aesthetics that create entirely different visual impacts.)
>
> 2. **Visual Structure & Alignment** (Score: 0–10): Does the placement of elements like images, buttons, and text boxes aligned similarly on both images? Do the sizes and aspect ratios of images, buttons, and text boxes appear consistent across both images? Do both UI images exhibit a similar level of visual harmony and balance in their design? (e.g., A score of 10 for identical visual structure and alignment, 5 for similar but not exact consistent, and 0 for completely different structure.)
>
> 3. **Textual and Content Consistency** (Score: 0–10): Do the font type, size, style, and weight of two UI images similar? Do the words and sentences match between the two UI images? (e.g., A score of 10 for complete uniformity in font type, size, style, weight across both UI images and identical text, 5 for consistency in font type and size but variations in style or weight, and 0 for wide disparities in font type, size, style, or weight, leading to a distinctly different textual appearance and content.)
>
> 4. **Interface Usability & Interactivity** (Score: 0–10): Do the user interface elements (like menus, buttons, and forms) share a clear and usable design language? Are interactive elements such as buttons, inputs, and navigation affordances visually identifiable and appropriately sized? (e.g., A score of 10 for clear and usable interactive elements, 6 for minor usability issues, and 0 for unclear or unusable UI designs.)

D.3 Contrastive Preference Ranking (CPR) Prompt

For pairwise comparison in CPR, we present two rendered UI images side-by-side and use the following prompt. To control for positional bias, we evaluate both orderings (A-left/B-right and B-left/A-right) and aggregate results.

> You are an expert UI designer. You are shown two UI images, Image A (left) and Image B (right), both generated from the same requirement. Compare them along the following four dimensions and for each dimension output which is better: A, B, or Tie.
>
> 1. **Color & Aesthetic Design**: Which UI has a more appealing and harmonious color scheme? Consider hues, saturation, gradients, and overall visual warmth.
>
> 2. **Visual Structure & Alignment**: Which UI has better element alignment, spacing, and layout balance? Consider the hierarchical organization and visual flow.
>
> 3. **Textual & Content Consistency**: Which UI has more appropriate text styling (font type, size, weight) and more accurate content relative to the requirement?

4. **Interface Usability & Interactivity**: Which UI has clearer interactive elements and better overall usability? Consider button clarity, tap target sizes, and navigational affordances.

Output format: a comma-separated list of 4 values, each being "A", "B", or "Tie". Example: A, Tie, B, A

D.4 General-Purpose MLLM Evaluation Prompts

For evaluating general-purpose MLLMs on our benchmark, we use standardized prompts with the full 80-category list.

Referring evaluation.

> Your task is to describe the given region in the image from three perspectives. First, decide the category of the given region from the following 80 categories: {full_category_list}. Second, describe the text content in the given region. Third, output the color of the text in hexadecimal format (e.g., #414141).
>
> Output in JSON format: {"category": "...", "text": "...", "text_color": ["..."]}. Do not return any other content.

Dense grounding evaluation.

> Your task is to detect all UI elements in the image. Classify each element using the following 80 categories: {full_category_list}. Output the bounding box in [x_min, y_min, x_max, y_max] format.
>
> Return in JSON format: [{"type": "...", "box": [x_min, y_min, x_max, y_max]}, ...]. Do not create new categories and do not return any other content.

GUI grounding evaluation.

> Output the bounding box corresponding to the following instruction: {instruction}. The output should just be the coordinates of a box, in the format [x0, y0, x1, y1]. If the task is infeasible, output [-1, -1, -1, -1].

E Dense Grounding Evaluation Protocol

Because dense grounding outputs are variable-length structured predictions, we use a dedicated set-matching evaluation protocol rather than confidence-based detection metrics.

Dense grounding evaluation requires matching a variable-length set of predicted bounding boxes to a variable-length set of ground-truth boxes, which is fundamentally a set-matching problem. Unlike standard object detection benchmarks that rely on confidence-score-based AP metrics, MLLMs do not produce per-box confidence scores. We therefore adopt a set-matching protocol:

Step 1: Parsing and filtering.

We parse the model's structured JSON output to extract predicted bounding boxes and their associated category labels. Predictions with malformed coordinates or categories outside the 80-class taxonomy are discarded.

Step 2: Hungarian matching.

For each image, we compute a cost matrix $\mathbf{C} \in \mathbb{R}^{M \times N}$ between M predicted boxes and N ground-truth boxes, where each entry is:

$$C_{j,i} = \begin{cases} 1 - \text{IoU}(\hat{b}_j, b_i) & \text{if } \hat{c}_j = c_i \\ 1.0 & \text{otherwise} \end{cases} \tag{14}$$

Here $\hat{c}_j$ and c_i denote the predicted and ground-truth category labels. We then apply the Hungarian algorithm to find the optimal one-to-one assignment that minimizes total cost.

Step 3: IoU thresholding.

A matched pair $(\hat{b}_j, b_i)$ is counted as a true positive only if both (a) the categories match ($\hat{c}_j = c_i$) and (b) $\text{IoU}(\hat{b}_j, b_i) \geq \tau$, where we use $\tau = 0.5$ as the default threshold. Unmatched predictions are false positives; unmatched ground-truth boxes are false negatives.

Step 4: Metric computation.

We compute:

- **Precision** $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$
- **Recall** $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$
- **F1**: computed per sample from its precision and recall, then averaged over the evaluation set.
- **mIoU**: computed over category-correct true-positive matched pairs, reflecting localization quality after semantic matching.

This protocol ensures fair comparison across models with different output formats.

F Additional Understanding Results

This section supplements the main paper with additional analyses for referring and grounding, including quantitative diagnostics and qualitative case studies.

F.1 Referring: Analysis Across Models

Figure 11 compares overall referring metrics across representative open/closed-source and UI-specific models. In addition to strict category accuracy, we report format validity, relaxed category accuracy, text matching/similarity, and color similarity to provide a more complete picture of model behavior.

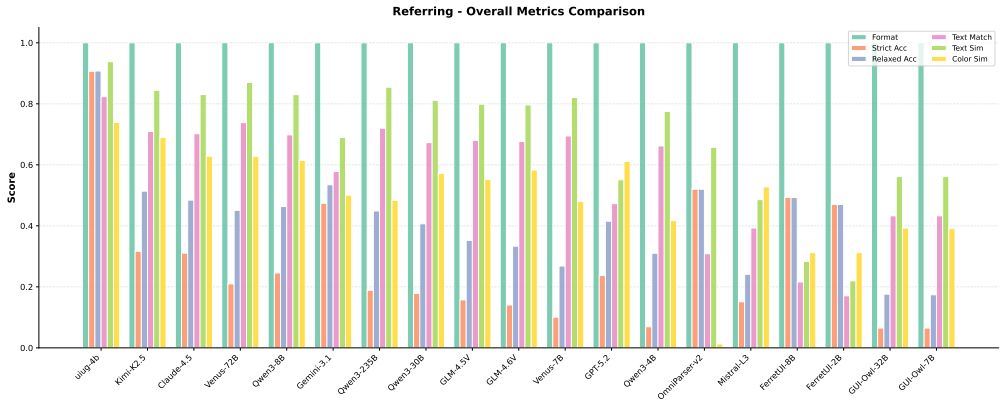


Figure 11: Overall metric comparison on the referring task across models.

F.2 Referring: Strict vs. Relaxed Type Matching

Although all evaluated MLLMs are prompted with the full 80-category taxonomy, strict type matching remains challenging in practice. Two factors are prominent: (1) limited instruction-following consistency for constrained category output, and (2) limited fine-grained visual discrimination between semantically adjacent UI types. Therefore, besides strict matching, we also evaluate a **relaxed matching** mode via post-processing, where visually and semantically similar predicted types are tolerated.

Figure 12 reports per-category precision under strict and relaxed settings. The relaxed protocol yields noticeable precision improvements for several general-purpose MLLMs, indicating that many strict-mode errors are near-miss confusions among closely related categories rather than complete perception failures. This analysis helps disentangle category granularity mismatch from core UI understanding capability.

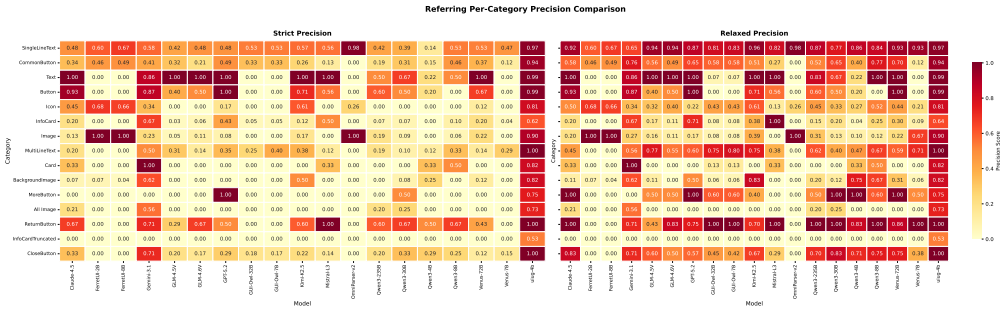


Figure 12: Per-category precision comparison under strict and relaxed type matching protocols.

F.3 Grounding: Analysis Across Models

We further compare grounding performance across models under both strict fine-grained labels and a relaxed macro-label protocol. Figure 13 reports four aggregate metrics: **F1@50**, **F1@75**, **macro_F1@50**, and **macro_F1@75**. Here, macro mode uses the broader 9-category taxonomy in our dataset, which is more tolerant to confusions among semantically similar fine-grained types.

Compared with strict fine-grained matching, macro-level evaluation raises scores for multiple general-purpose MLLMs, indicating that part of their errors come from fine-grained type ambiguity rather than complete localization failure. This is consistent with the practical difficulty of dense UI grounding: models may localize a region reasonably but still miss the exact fine category.

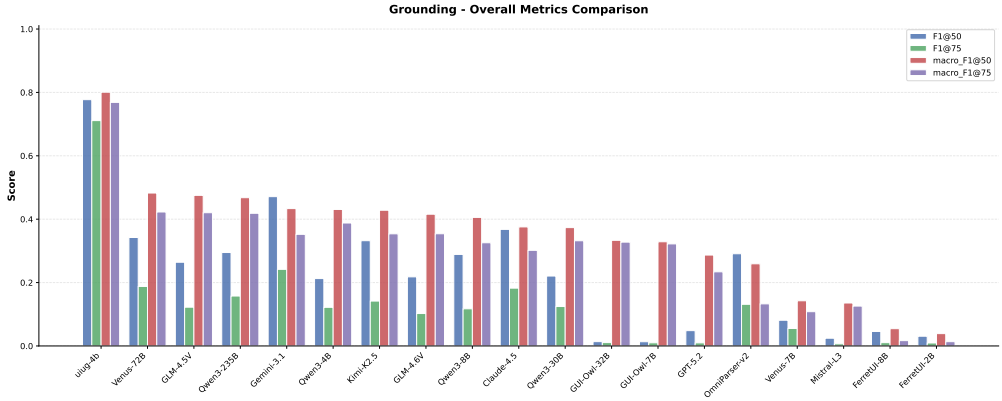


Figure 13: Overall grounding metric comparison across models under strict and macro-label protocols (F1@50/F1@75 and macro_F1@50/macro_F1@75).

Figure 14 presents per-category F1@50 differences. Stronger grounding models (e.g., Claude Sonnet 4.5, Gemini 3.1 Pro, and Kimi K2.5) generally achieve better detection on frequent and visually salient categories such as *Icon*, *Button*, and *Text*. However, performance degrades substantially on complex or easily confused UI categories (e.g., nested card-like structures, fine-grained functional controls, and visually similar component variants). This suggests that the primary bottleneck of current MLLMs is still **fine-grained UI semantic understanding and category discrimination** under dense layouts, rather than only coarse object localization.

F.4 Grounding: Visual Comparison of Results

Figure 15 presents qualitative comparisons of grounding results across representative models on two dense UI cases. Our model demonstrates stronger category recognition and tighter bounding boxes across both examples.

From the qualitative comparisons, we observe three consistent patterns. **First**, several baseline models are weaker in *bounding-box localization precision*: for example, GPT-5.2 often exhibits no-

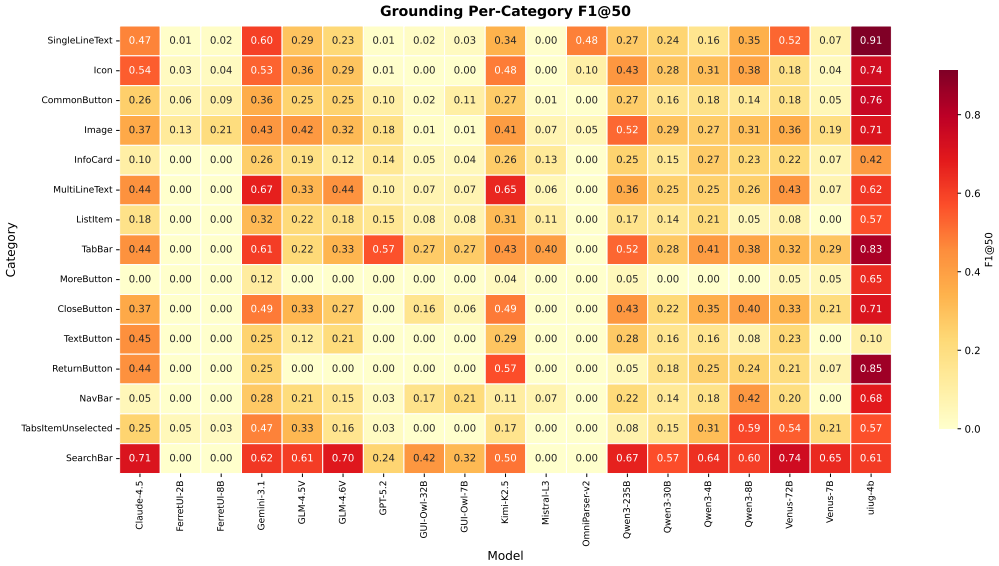


Figure 14: Per-category grounding F1@50 comparison across models. Common categories are generally easier, while complex or confusable UI categories remain challenging for most baselines.

ticeable spatial offsets, while other models also show imperfect alignment on narrow regions such as top navigation bars and bottom icon areas. **Second**, baseline predictions show more *missed detections*, including both high-level composite components (e.g., outer boxes of product cards) and fine-grained inner elements (e.g., small icons inside cards). **Third**, there are recurring *category confusions* for function-oriented controls, such as failing to correctly detect or classify return/back buttons. These failure modes are reduced in UI-UG 2-4B, which yields more complete and structurally consistent grounding results.

F.5 Grounding: GUI Grounding Per-Category Analysis

Table 8 reports UI-UG 2-4B's GUI grounding results on both ScreenSpot-V1 Mobile and ScreenSpot-V2 Mobile. We group the results into Icon and Text categories and report the corresponding Overall accuracy per dataset. Text is consistently slightly higher than Icon, and the overall accuracy is 0.902 on V1 and 0.929 on V2.

Table 8: UI-UG 2-4B GUI grounding accuracy on ScreenSpot mobile benchmarks by element type.

Dataset	Icon	Text	Overall
ScreenSpot-V1 Mobile	0.887	0.916	0.902
ScreenSpot-V2 Mobile	0.912	0.944	0.929

F.6 Attention Visualization Analysis

In the main paper, we already presented an attention-mechanism experiment. To provide a clearer and more interpretable view of that behavior, we further conduct attention visualization analysis in this appendix.

Figure 16 focuses on **referring-task** cases and shows a clear task-conditioned attention shift. In referring-text examples, high-response regions are concentrated around the target text spans; in referring-button examples, attention moves to the corresponding interactive button areas. This qualitative evidence supports the same conclusion as our quantitative results: the model can adapt its visual focus according to referring intent, which is critical for stable category grounding and precise target localization in UI understanding.

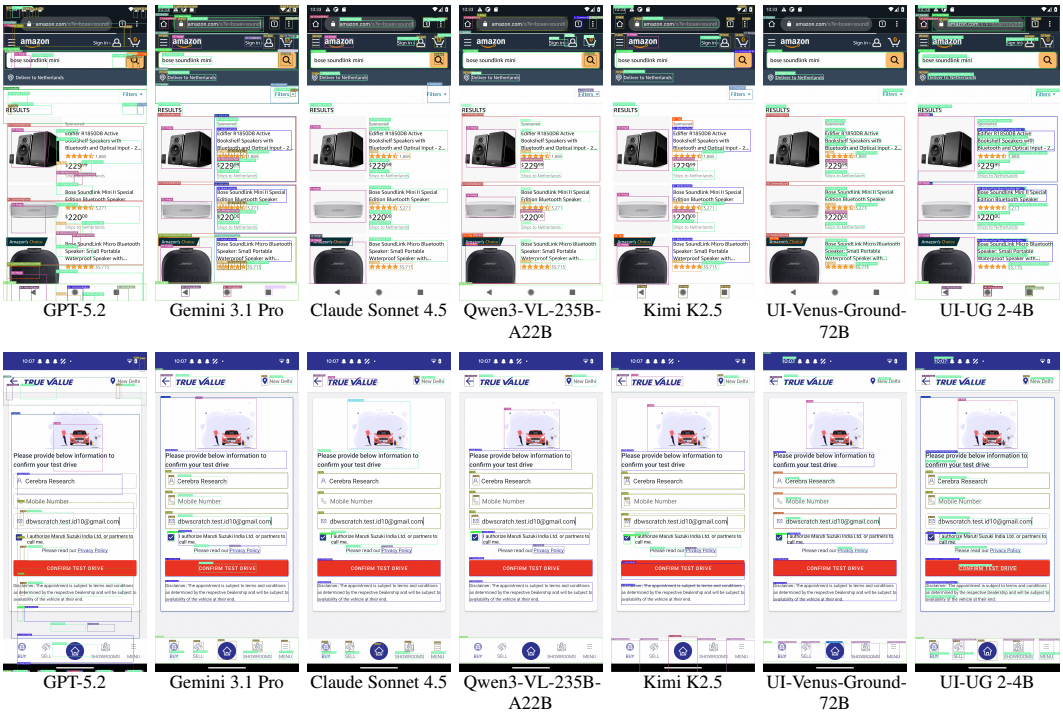


Figure 15: Visual comparison of grounding results on two dense UI examples (2×7). Each row follows the same model order: GPT-5.2, Gemini 3.1 Pro, Claude Sonnet 4.5, Qwen3-VL-235B-A22B, Kimi K2.5, UI-Venus-Ground-72B, and UI-UG 2-4B (ours).

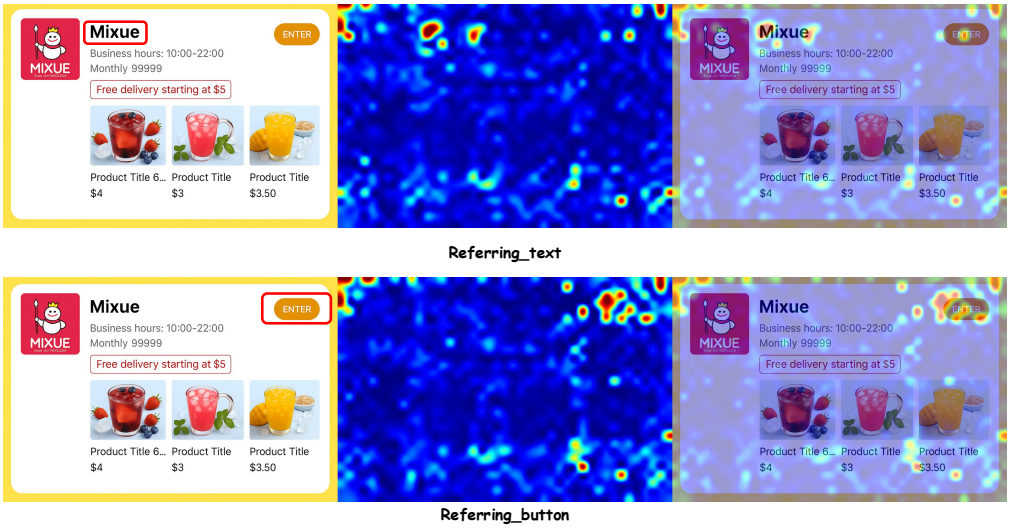


Figure 16: Attention maps demonstrate task-dependent focus, with highlights concentrated on the corresponding target regions.

G Additional Generation Results

This section provides supporting analyses for generation quality, judge reliability, and qualitative comparisons beyond the main-paper table.

G.1 LLM-as-a-Judge Reliability Analysis

We evaluate judge reliability on 100 generated UI cases. For each case, three professional UI designers provide scores on four dimensions (Color & Aesthetic, Structure & Alignment, Textual & Content Consistency, and Interface Usability & Interactivity), and the averaged human score is used as the reference value. The designers were recruited based on prior mobile UI design experience, scored anonymized model outputs independently under the same rubric as the MLLM judges, and were compensated according to their standard annotation rate. Table 9 compares four MLLM judges (Qwen3-VL-235B-A22B, GPT-5.2, Claude Sonnet 4.5, and Gemini 3.1 Pro) against this reference using score deviation (Δ) and variance.

Table 9: LLM-as-a-Judge reliability on 100 samples. Δ is the mean absolute deviation from human reference (lower is better). Variance is computed on residuals against the human reference.

Judge Model	Δ_{Col}	Δ_{Str}	Δ_{Txt}	Δ_{Int}	Δ_{Overall}	**Variance**
Gemini 3.1 Pro	**0.07**	**0.08**	**0.09**	**0.08**	**0.08**	**0.032**
Qwen3-VL-235B-A22B	0.15	0.17	0.13	0.20	0.17	0.046
GPT-5.2	0.12	0.11	0.14	0.12	0.13	0.049
Claude Sonnet 4.5	0.21	0.24	0.23	0.26	0.24	0.050

Overall, the four judges show similar reliability ranges. Their absolute deviations are moderate and residual variance remains controlled, indicating that MLLM-based aesthetic evaluation is reasonably stable. In raw-score inspection, judges tend to assign slightly higher scores than human references. Gemini 3.1 Pro performs best. GPT-5.2 and Qwen3-VL-235B-A22B are close in overall deviation with comparable variance, while Claude Sonnet 4.5 shows a larger average deviation in this evaluation.

Although Gemini 3.1 Pro obtains the best evaluation performance, we finally use Qwen3-VL-235B-A22B for GRPO for practical repeated evaluation through the separately provisioned judge service.

H Inference Efficiency

To complement quality results, this section reports practical latency-oriented efficiency analysis with measured values distinguished from scaling estimates.

H.1 Inference Time Estimation for UI Generation

To provide a size-aware efficiency reference, Table 10 reports estimated relative latency across representative open-source model scales for UI generation. We do not include closed-source API models in this comparison for two reasons: (1) the goal here is to analyze the relationship between model size and speed under controlled assumptions, and (2) API serving speed from closed-source providers is often unstable due to external service variance.

UI-UG 2-4B latency is measured on 100 sampled UI-generation cases with an average output length of 894 tokens. For larger open-source baselines, we report a size-aware estimate under a 16×A100 inference setting, using **active parameters** for MoE models and normalizing UI-UG 2-4B (4B) to **1.00**×. These estimates contextualize scaling trends rather than replace hardware-specific deployment benchmarking.

Table 10: Estimated UI generation latency using a size-based scaling factor for larger open-source baselines.

Model	Size	Size-Based Factor	Estimated Time (s)
UI-UG 2-4B	4B	1.00×	0.9
Qwen3-VL-30B-A3B	A3B	0.75×	0.7
UI-Venus-Ground-7B	7B	1.75×	1.6
Qwen3-VL-8B	8B	2.00×	1.8
UI-Venus-Ground-72B	72B	18.00×	16.0
Qwen3-VL-235B-A22B	A22B	5.50×	4.9

Although Qwen3-VL-30B-A3B has a much larger total parameter count than UI-UG 2-4B, its active parameter size (A3B) is smaller than the dense 4B baseline, which explains the lower estimated latency under the active-parameter approximation.

H.2 Progressive Rendering Illustration

Figure 17 illustrates progressive rendering of generated UI specifications.

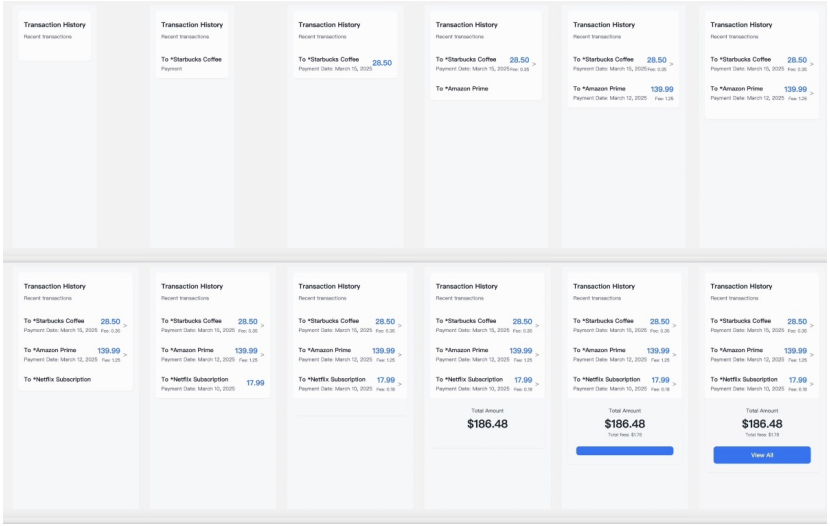


Figure 17: Progressive rendering of generated UI specifications.